Improving USB-C DisplayPort Support on M1/M2 Macs in Asahi Linux

Senior Design Project

Ben Deutsch, Eric Veilleux, Dennis Gega, Mustafa Maras, Judah Friedman, Nakul Ramaswamy

October 2026

School of Computing
College of Engineering
University of Connecticut
Storrs, Connecticut

Word Count: (Count)

Contents

1 Introduction **2**
1.1 Project Overview . . . 2
1.2 Motivation . . . 2
1.3 Technical Background . . . 2
1.4 Current State and Candidate Problems . . . 3
1.5 Scope . . . 3

2 Project Plan and Risk Analysis **4**
2.1 Objectives . . . 4
2.2 Timeline . . . 4
2.3 Risk Analysis . . . 5
2.4 Deliverables . . . 6

3 References **7**

1 Introduction

1.1 Project Overview

This senior design project will investigate and improve hardware support in Asahi Linux on Apple Silicon M1 and M2 Macs, with a primary focus on the USB-C display path and a secondary focus on USB4/Thunderbolt functionality. The long-term objective is hardware feature parity with the capabilities available under macOS, but the project will define success in terms of measurable contributions to the existing Asahi Linux kernel rather than requiring complete parity within one academic year.

The work will build on the Asahi Linux `fairydust` branch, which contains experimental USB-C DisplayPort Alternate Mode support. Current project research indicates that basic external display output already works in some configurations, but the branch remains work in progress. Important remaining problems include DisplayPort Alt Mode negotiation, hot-plug and reconnect reliability, link training, monitor and adapter compatibility, suspend/resume behavior, display routing, DCP firmware handling, USB4/Thunderbolt, and PCIe tunneling.

The project is therefore not starting from a blank implementation. Instead, the team will work within an active open-source kernel codebase, reproduce existing limitations, compare Linux behavior against working macOS behavior, identify missing or incorrect hardware interactions, and implement targeted improvements.

1.2 Motivation

External display and high-speed USB-C connectivity are important features on both laptops and desktops. On the M1 and M2 systems considered by this project, these features depend on several Apple-specific hardware blocks whose interfaces were not originally documented for Linux. Asahi Linux support has consequently required substantial reverse engineering. The current state is far enough along that basic functionality exists, but not yet far enough along that all monitors, adapters, reconnect sequences, power transitions, and Thunderbolt use cases behave reliably.

This creates a useful senior design problem for three reasons. First, the remaining failures are concrete and experimentally testable. For example, current research identifies a reconnect failure in which a display works initially but may remain black after being unplugged and reconnected. It also identifies cases where Linux successfully detects a monitor, reads its EDID, enumerates modes, configures a DisplayPort link, and completes a DCP modeset, yet the monitor still remains black. These behaviors provide observable baselines against which improvements can be measured.

Second, the work combines several areas of computer engineering and systems software: Linux kernel development, device drivers, USB-C protocol handling, DisplayPort link training, firmware communication, power management, hardware tracing, and reverse engineering. The project therefore has substantial technical depth while still producing user-visible outcomes.

Third, the work can contribute directly to an existing open-source ecosystem. The `fairydust` branch is explicitly intended for continued development and validation, and the team can structure patches and documentation so that useful results are suitable for discussion with the Asahi Linux community.

1.3 Technical Background

The USB-C and display path on Apple Silicon involves several interacting hardware and software components. Team research on the current kernel identifies the Apple Type-C PHY (ATCPHY) as a central component. The ATCPHY supports USB 2.0, USB 3.x, DisplayPort, and USB4/Thunderbolt connectivity over the Type-C ports. It multiplexes these protocols and also provides reset control for the attached DWC3 USB controller.

The project research further identifies four important blocks in the current DisplayPort path: DCP (Display Coprocessor), DPXBAR (Display Crossbar), ATCPHY (Apple Type-C PHY), and ACE (USB-C Power Delivery controller). Each required reverse-engineered Linux support. For M1 through base M3 systems, the USB-C controller is an Apple-specific Texas Instruments controller referred to as CD3217 or ACE2, which participates in negotiation when a device is connected.

At a high level, the relevant responsibilities can be summarized as follows:

Component	Relevance to Project
ACE / CD321x	USB-C and Power Delivery negotiation, cable/orientation state, and entry into alternate modes.
ATCPHY	Physical-layer configuration and protocol multiplexing for USB3, DisplayPort, and USB4/Thunderbolt.
DWC3	USB controller coordinated with the Type-C/PHY stack.
DPXBAR	Routes display signals through the Apple display subsystem.
DCP / DPTX	Handles important display firmware and mode-setting behavior.
Linux DRM / Type-C stack	Provides the kernel frameworks through which connectors, modes, Alternate Mode behavior, and display state are represented.

Table 1: Major components relevant to the proposed work.

Because these components interact, a visible failure does not necessarily identify the faulty layer. A black screen after reconnection could originate in DCP state management, USB-C negotiation, ATCPHY configuration, link training, routing, or another part of the sequence. The project will therefore use both software debugging and hardware tracing to isolate failures before modifying code.

1.4 Current State and Candidate Problems

The team's current research identifies several promising areas for contribution.

Hot-plug and reconnect reliability. A known Fairydust failure occurs when a display works on initial connection but remains black after an unplug/reconnect cycle. Two distinct failure classes have been identified. One involves DCP being marked as permanently crashed after a shutdown operation exceeds a timeout. Another involves the CD321x/USB-C controller failing to re-enter DisplayPort Alternate Mode after certain failures, leaving power or USB partially functional without a valid DisplayPort negotiation sequence.

DisplayPort link training. A second issue occurs when software-side setup appears successful but no image is produced. Current research suggests that voltage swing and pre-emphasis values requested by the DisplayPort sink may be stored by the driver without being fully applied to the ATC PHY. This could explain why behavior varies across monitors, cables, DisplayPort versions, and bandwidth requirements.

Adapter and monitor interoperability. Different behavior from different adapters has been documented on the same machine and monitor. In one case, a UGREEN adapter was stable while a VIA Labs-based hub repeatedly connected and disconnected after successful detection and EDID reads. Incomplete handling of a DCP firmware event observed with an Apple Studio Display has also been identified.

Suspend and resume. DisplayPort suspend/resume remains incomplete. Some related hardware is currently forced to remain powered, while the desired behavior is to power down the display path, resume the system, reinitialize the PHY/display path, and restore the external monitor automatically.

USB4/Thunderbolt and PCIe tunneling. Active USB4/Thunderbolt development for Apple Silicon has been identified and basic USB4 infrastructure, interaction with the Linux Thunderbolt software connection manager, initial XDomain support, and USB3 tunneling are available in recent work. Full DisplayPort tunneling, PCIe tunneling, complete hot-plug/device compatibility, and performance optimization remain incomplete. PCIe tunneling is therefore an appropriate stretch goal rather than a core success requirement.

1.5 Scope

The primary scope of this project is M1 and M2 Mac hardware, with emphasis on USB-C DisplayPort reliability. The team will prioritize problems where a reproducible failure can be traced, understood, modified in the kernel, and validated experimentally. The project will not attempt to solve every remaining Asahi Linux hardware limitation.

Multi-port DisplayPort routing is particularly important but is not proposed as the primary target. Team research indicates that this area requires broader architectural work, including per-port DRM connectors, dynamic DCP/DCPEXT assignment, crossbar routing, power-domain management, and Device Tree changes, and that upstream Asahi developers are already planning their own solution. The team may validate upstream multi-port work if it becomes relevant, but will avoid duplicating an actively owned upstream redesign.

Similarly, full USB4 and PCIe tunneling will remain stretch goals. This keeps the project achievable even if low-level reverse engineering takes longer than expected while still allowing the team to pursue more advanced feature parity if the primary DisplayPort goals are completed early.

2 Project Plan and Risk Analysis

2.1 Objectives

As outlined in the introduction, our main objective is to make contributions to the asahi linux kernel. To complement the code changes (and to adhere to university standards for senior design) we will also consider writing at least 15 pages of documentation outlining our debugging process, implementation details, and results as a main objective of the project.

Secondary objectives include gaining experience within the open-source community. This includes working on large codebases with code written by third parties, communicating with maintainers, and preparing and submitting patches for review by the Asahi Linux community, initially targeting the `fairydust` branch where appropriate and ultimately upstream Linux where feasible.

Finally, we aim to develop professional engineering habits by emulating the practices of real-world software engineering teams. Specifically we will practice Agile workflows, participate in stand up meetings, and communicate clearly both within the team, and with our supervisors.

2.2 Timeline

We begin by emphasizing that the following timeline is tentative. The team reserves all rights to make changes to the timeline, including moving deadlines, and adding/removing phases.

October '26 - Phase 0 To begin making meaningful progress, we must first be able to set up our development environment, and understand the codebase.
Phase Goals

- Build and boot the `fairydust` branch on each test machine, pinned to a specific commit.
- Set up a debugging workflow: potentially using m1n1 in order to poke registers.
- Read through the relevant driver code and write short internal notes on how the pieces connect.
- Set up the repo workflow: branch strategy, commit conventions, and shared notes for logs and findings.

November '26 - Phase 1 We can now get a baseline by testing the hardware and creating a repeatable set of failures.
Phase Goals

- Standardize the testing process, this includes using same cables, monitors, adapters etc.
- Write scripts that automate logging and trace collection around standardized, manually performed unplug/replug tests.
- Run each configuration enough times to estimate failure rates.
- Categorize failures by source of the issue (DCP Timeout, Driver bugs etc).

Dec. 2026–Feb. 2027 - Phase 2 Fix the problems identified in Phase 1 by breaking them into small, reviewable changes.
Phase Goals

- Break each issue into small, self contained pieces in order to simplify communication and planning.
- Sort the fixes by importance and feasability, and plan more granular deadlines.
- Test each fix on all testing hardware, keeping track any potential differences between setups.
- Hold code reviews with the team and keep track of what each change fixes.

Mar '27 - Phase 3 Buffer for implementation phase. Then validate our changes quantitatively by repeating tests.
Phase Goals

- Wrap up any left over tasks from the implementation stage.
- Compare the baseline code with our branch, using the logging and trace-collection scripts from Phase 1.
- Repeat these tests in order to calculate revelant statistics (reconnect rate, failure counts).
- Log failures and shortcomings of our changes.

Apr '27 - Phase 4 Prepare patches for upstream review, then prepare the final demo and report.
Phase Goals

- Clean up commit history by rebasing, squashing changes and conforming to kernel guidelines.
- Publish technical documentation in the team's public repository and submit it for possible inclusion in Asahi Linux documentation, subject to maintainer review and acceptance.
- Share our progress with the asahi community, asking for feedback.
- Prepare final presentation, including live demo of usb functionality.

Stretch Goals Note that we have not included stretch goals in the current timeline. This coincides with our decision to not work on these goals until the functionality we deem to be "core" to meeting our objectives is completeed. If we are ahead of schedule, we will firstly look at refining our patches, and communicating with the maintainers in order to get our code upstream.

If we do have time for stretch goals, these would likely be fit in between Phase 3 and 4, and potentially be executed in parallel with each specific set of phase goals.

2.3 Risk Analysis

Given that we are working on a large open source project, specifically on drivers for propietary hardware there is significant potential for risk. Therefore we have divided potential issues by category, and outline mitigation strategies for each case.

Technical Risks .

Risk	Mitigation	Backup
Diagnosis process takes longer than expected	Reach out to developers who reverse engineered the hardware	Limit our fixes to one failure type
Failure class is hard to reproduce	Automate testing in order to reproducte each issue a large amount of times	Prioritize issues that we are able to reliably replicate, document why we aren't able to reproduce the others
A fix works on one testing setup, but affects functionality on different setups.	Keep changes small, and test frequently.	Revert changes, document unusual behavior.
Root cause is outside of scope (i.e. hardware bug)	Document the limitations, try to hack around the issue	Get hired as an hardware engineer at Apple and implement the fix

External .

Risk	Mitigation	Backup
Other developers fix the issue before the team	Communicate with the community, "mark" our territory.	Pivot to extending their fixes, or move to a lower priority fix
Upstream does not reciprocate communication or respond to our fixes	Use the other metrics defined in 2.1 as measure of success	Publish the changes in our fork of the kernel

Hardware .

Risk	Mitigation	Backup
Issue not reproducible on our hardware	Prioritize fixes that are reproducible	Document the issue and move on to different failure
Test machine becomes unbootable, or is damaged by kernel	Have a recovery procedure, keep a dedicated backup machine	Shared access to backup machine, purchase new hardware

Team and schedule .

Risk	Mitigation	Backup
Lack of technical experience significanlty slows down development	Members return to Phase 0, or pair up on issues	Move teammates to testing/benchmarking as needed
Scope creep	No stretch work until core objectives are met	Cut stretch goals as needed
Scheduling conflicts around major deadlines	Plan ahead of time in order to prevent this possibility, built in buffer in timeline.	Reassign tasks, schedule for maximal overlap

In the likely case that unforeseen risks do happen, we will commit to openly communicate the issues we encounter and come up with strategies in order to mitigate the issue, or find backup options to keep the process on its tracks.

2.4 Deliverables

We conclude this draft by outlining our plan concerning the concrete deliverables of this project. We will have reports on the baseline failure rates of the kernel, compared to our patched version. These tests will be completend using custom build automation and logging scripts, which will be fine tuned for our specific use-cases.

We will also compile a series of reports, beginning from a failure catalogue where we separate each failure by class, describe conditions needed in order to recreate, and list hardware it happens on. We will also provide a Diagnosis report where we confirm root causes, or use evidence in order to point to supect causes. Finally we will write a final report, and create an interactive demo to expose our work to the public on demo day.

Of course we will also look to deliver hotfixes to the issues identified by the team, potentially having them available on upstream asahi. Whether or not the patch is upstreamed, we aim for our updates to be upstream-ready which means conforming to best practices and repository guidelines. This will be paired with documentation as we have discussed beforehand.

In terms of strech goals, we aim to provide full USB4 and PCIe tunneling, this would allow for external gpu usage, which is functionality not natively supported by Apple and would be exclusive to asahi. Additionally we could also target multi-port displays, as it is a known issue that the current code selects one port for ouput, and "locks" all the others. If we do achieve these goals the demo could involve playing modern videogames on Apple hardware running linux, with other applications open on other desktops.

3 References

[1] Asahi Linux. *AsahiLinux/linux: Asahi Linux downstream kernel tree (includes the fairydust branch)*. GitHub repository. https://github.com/AsahiLinux/linux. Accessed October 4, 2026.

[2] Asahi Linux. *m1n1: Apple Silicon bootloader and experimentation tool*. GitHub repository. https://github.com/AsahiLinux/m1n1. Accessed October 4, 2026.

[3] Asahi Linux. *Asahi Linux Documentation*. https://asahilinux.org/docs/. Accessed October 4, 2026.

[4] Asahi Linux. *Tethered boot setup on macOS*. Asahi Linux documentation wiki. https://github.com/AsahiLinux/docs/wiki/Tethered-boot-setup-on-macOS. Accessed October 4, 2026.

[5] Asahi Linux. *Progress Report: August 2021*. August 2021. https://asahilinux.org/2021/08/progress-report-august-2021/. Accessed October 4, 2026.

[6] Asahi Linux. *Progress Report: Linux 6.19*. February 2026. https://asahilinux.org/2026/02/progress-report-6-19/. Accessed October 4, 2026.

[7] mwmetcalf1. *[fairydust] M2 Max: HDMI detects but no output, USB-C DP not working*. GitHub issue #456, AsahiLinux/linux. https://github.com/AsahiLinux/linux/issues/456. Accessed October 4, 2026.

[8] Linux Kernel Developers. *USB Type-C connector class*. Linux kernel documentation. https://docs.kernel.org/driver-api/usb/typec.html. Accessed October 4, 2026.

[9] Linux Kernel Developers. *USB4 and Thunderbolt*. Linux kernel documentation. https://docs.kernel.org/admin-guide/thunderbolt.html. Accessed October 4, 2026.